SAPIENZA
UNIVERSITÀ DI ROMA

# Implementation of the Einstein sum

Department of Computer Science
Applied Computer Science and artificial intelligence

**Domenico Menale**
ID number 2049488

Advisor
Prof. Maria De Marsico

Co-Advisor
Dr. Diego Bellani

Academic Year 2025/2026

Thesis not yet defended

---

**Implementation of the Einstein sum**
Bachelor degree. Sapienza University of Rome

© 2026 Domenico Menale. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: menale.2049488@studenti.uniroma1.it

*Da grandi poteri,*
*derivano grandi responsabilità.*

## Abstract

This work explores the different possible implementation of the Einstein sum, a mathematical notation used in many fields of science, such as physics and engineering, to represent the summation of products of tensors. The Einstein sum is a powerful tool for simplifying complex mathematical expressions and can be used to derive important results in various fields. In this work, we will discuss different ways to implement the Einstein sum, including its use in tensor calculus. We will also explore the advantages and disadvantages of each implementation.

# Contents

**1 Introduction to Einstein Summation and Tensors** **1**
1.1 Tensors memory layout and access patterns . . . . . 1
1.2 Einsum in the Architecture of Neural Networks . . . . . 2
1.2.1 Linear Layers and Batch Processing . . . . . 2
1.2.2 Broader Applications: From Fluids to Architecture . . . . . 3
1.3 From Mathematical Notation to algorithm . . . . . 4
1.3.1 The Compilation Pipeline . . . . . 5
1.3.2 Execution Strategies and Algorithmic Diversity . . . . . 5

**2 Implementations of the Einstein sum** **7**
2.1 Generalized Tensor Contraction via Odometer Iteration . . . . . 7
2.1.1 The Technical Architecture . . . . . 7
2.1.2 Execution and Performance Characteristics . . . . . 8
2.1.3 Summary of the Generalized Engine . . . . . 8
2.2 Just-In-Time (JIT) Code Generation and AOT Specialization . . . . 8
2.2.1 Optimization Mechanisms . . . . . 9
2.2.2 Performance and Memory Trade-offs . . . . . 9
2.2.3 Summary of the JIT Engine . . . . . 10
2.3 Canonical Contraction through Permutation and Flattening . . . . . 10
2.3.1 The Transformation Pipeline . . . . . 10
2.3.2 Performance and Memory Trade-offs . . . . . 11
2.3.3 Summary of the Permute Logic . . . . . 11
2.4 BLAS-Accelerated Pattern Recognition and Greedy Dispatch . . . . 11
2.4.1 The Pattern Hierarchy . . . . . 12
2.4.2 Intelligent Dispatch Logic . . . . . 12
2.4.3 Future Work: Stride-Based Optimization . . . . . 12
2.4.4 Summary of the BLAS Engine . . . . . 13

# Chapter 1

# Introduction to Einstein Summation and Tensors

For our work we need to first define what is a tensor and what is the Einstein summation convention. Tensors are a widely used representations of multidimensional data in scientific and engineering applications. [4] In the physical world—specifically in architectural engineering and structural analysis—tensors represent physical quantities that are independent of any specific coordinate system. For example, the stress tensor $\sigma_{ij}$ describes the internal forces within a solid body. To calculate the interaction between these multi-dimensional objects, we can use a robust notation: the **Einstein Summation Convention**.

The summation convention was introduced by Einstein in 1916. It implies that whenever an index in a term is repeated, once as a superindex and once as a subindex, a summation over that index is understood. With these conventions, matrix vector multiplication is written. [7]

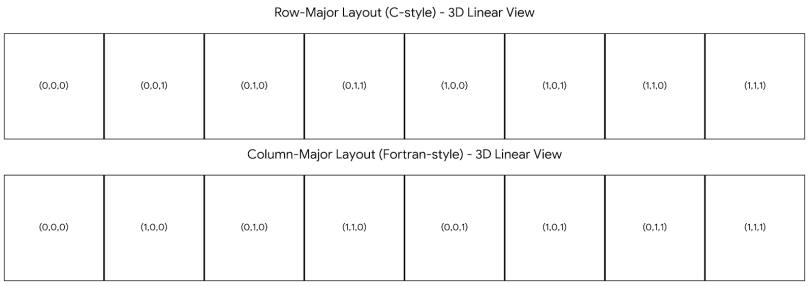
Consider the transformation of a vector $v$ by a matrix $M$:

$$y_i = \sum_{j=1}^{n} M_{ij} v_j \quad \longrightarrow \quad y_i = M_{ij} v_j \tag{1.1}$$

This notation formalizes the **contraction** process: a linear mapping that reduces the rank of a tensor by summing over one or more indices.

## 1.1 Tensors memory layout and access patterns

In the implementations that we will see in the next chapter, tensors are stored in contiguous memory buffers. The way these multi-dimensional arrays are laid out in memory is crucial for performance. The two most common layouts are **Row-Major** (C-style) and **Column-Major** (Fortran-style). In a row-major layout, the last index changes fastest, while in column-major, the first index changes fastest. For example, a 3D tensor $T_{ijk}$ with shape $(2, 2, 2)$ would be stored as:




The choice of layout affects the **memory access patterns** during tensor contractions. Accessing elements that are stored contiguously in memory leads to better cache performance and faster execution. For instance, in a row-major layout, iterating over the last index $k$ of $T_{ijk}$ will be more efficient than iterating over the first index $i$. Furthermore we can define the **stride** of a tensor as the number of elements to skip in memory to move from one index to the next. For a tensor with shape $(d_0, d_1, d_2)$ in row-major order, the strides would be:

$$\text{stride}_0 = d_1 \cdot d_2, \quad \text{stride}_1 = d_2, \quad \text{stride}_2 = 1 \tag{1.2}$$

and the offset as

$$\text{Offset} = i \cdot \text{stride}_0 + j \cdot \text{stride}_1 + k \cdot \text{stride}_2 \tag{1.3}$$

Understanding these concepts allows us to implement efficient tensor operations, as they directly impact the performance of algorithms that rely on tensor contractions, such as those expressed in Einstein summation notation.

## 1.2 Einsum in the Architecture of Neural Networks

In the modern computational landscape, tensors are the primary data structure for **Deep Learning**.

### 1.2.1 Linear Layers and Batch Processing

In modern deep learning architectures, a Linear Layer serves as an affine transformation that projects an input tensor into a new feature space via a learned weight matrix. This operation is fundamentally a **tensor contraction**, which can be concisely expressed using the Einstein summation convention. Consider a weight matrix $W$ with dimensions $(I, O)$, where $I$ is the number of input features and $O$ is the number of output features. If we process a single vector $x$ of size $I$, the

operation is a standard matrix-vector product. However, with a batch size $B$, our input becomes a 2D tensor $X \in \mathbb{R}^{B \times I}$.

Using the `einsum` notation, the operation is expressed as:

$$Y_{bo} = X_{bi} W_{io} \tag{1.4}$$

This specific notation provides several computational advantages:

**Feature Mapping:** The index $i$ is the "contraction dimension." It represents the internal connection where each input feature is weighted and summed to contribute to an output feature.

**Batch Preservation:** The index $b$ is a "broadcast index." It remains untouched by the summation, meaning the same weights $W$ are applied identically to every sample in the batch.

**Parallelism:** By representing the batch as a tensor dimension, the underlying `einsum` engine can dispatch the entire operation as a single `GEMM` (General Matrix Multiply) call.

Without the batch dimension, the hardware would have to launch $B$ separate, smaller kernels, leading to significant overhead and under-utilization of the high-speed memory bus. Thus, the Einstein notation does not just describe the operations, it defines the **memory access pattern**.

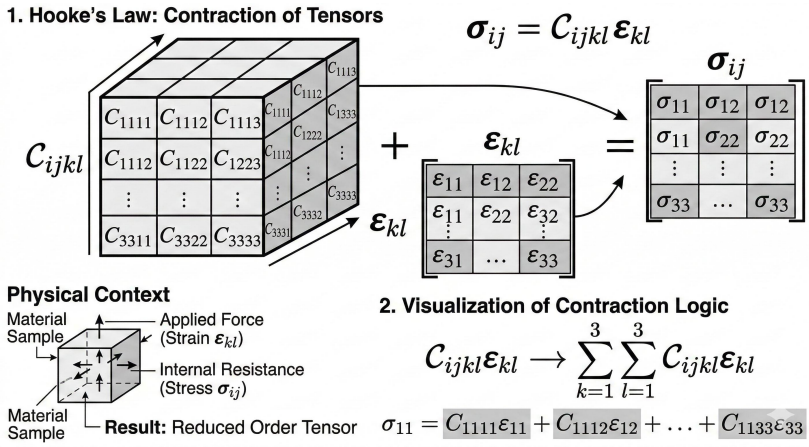
### 1.2.2 Broader Applications: From Fluids to Architecture

The utility of the Einstein summation convention extends far beyond the optimization of neural networks; it can describe physical systems where multi-axial forces interact.

#### Continuum Mechanics and Structural Elasticity

In structural engineering, the linear elastic response of a material—such as a reinforced concrete boundary—is governed by the generalized **Hooke's Law**. This relationship defines the second-order stress tensor $\sigma_{ij}$ as the result of a contraction between a fourth-order elasticity (stiffness) tensor $\mathcal{C}_{ijkl}$ and a second-order strain tensor $\varepsilon_{kl}$ [3]:

$$\sigma_{ij} = \mathcal{C}_{ijkl} \varepsilon_{kl} \tag{1.5}$$




This operation involves a double contraction over indices $k$ and $l$, effectively reducing a system of 81 potential coefficients in the elasticity tensor to a 9-component stress representation. Within the framework developed in this work, this is concisely expressed as `einsum('ijkl,kl->ij')`. Such a formulation can be used to simulate how external loads or thermal expansion distribute internal forces across complex architectural boundaries [2].

### Fluid Dynamics and Wind Modeling

Similarly, modeling the environmental conditions of a built environment—such as the wind pressure distribution within a central courtyard—requires the **Navier-Stokes equations**. In their tensor form, the advection term describes how the velocity field $u$ interacts with its own gradient [1]:

$$\rho\left(\frac{\partial u_i}{\partial t} + u_j\frac{\partial u_i}{\partial x_j}\right) = -\frac{\partial p}{\partial x_i} + \mu\frac{\partial^2 u_i}{\partial x_j \partial x_j} \tag{1.6}$$

The term $u_j\frac{\partial u_i}{\partial x_j}$ represents a tensor contraction between the velocity vector and the velocity gradient tensor. In a computational fluid dynamics (CFD) context, an `einsum` implementation facilitates the calculation of these gradients across a discretized 3D grid, mapping the local flow direction to the resultant acceleration vector through the contraction `einsum('j,ji->i')`.

## 1.3 From Mathematical Notation to algorithm

The translation of a symbolic string like `"ij,jk->ik"` into machine code is not a trivial task. The implementation must bridge the gap between high-level logic and **Low-Level Stride Arithmetic**.

### 1.3.1 The Compilation Pipeline

When a user calls an `einsum` function, the system undergoes several stages of translation:

**Parsing** The string is analyzed to create bitmasks representing index occupancy.

**Contraction Mapping** The engine identifies which indices are shared (summation) and which are unique (free).

**Memory Stride Calculation** To access data in a flat memory buffer, the engine calculates the *stride*—the number of elements to skip to reach the next index in a given dimension.

### 1.3.2 Execution Strategies and Algorithmic Diversity

Since the Einstein notation is so expressive, no single algorithm can handle every case with peak efficiency. For this reason we present different implementations that fit different use cases, strarting with a general unoptimized algorithm and moving towards better performing options.

**Generalized Odometer Interpretation:** The most flexible approach is the Odometer-based iteration. This strategy treats the $N$-dimensional space as a single flat counter. It identifies every unique index in the notation (both free and summed) and builds a virtual "odometer", an instrument used to measure distances traveled by cars, where each gear corresponds to a dimension size. Every time the odometer ticks the counter advances by one in the lowest dimension, and when it overflows, it resets to zero and advances the next gear or, in our case, the next dimension in the tensor. In computational terms, this allows us to simulate a nested loop structure of arbitrary depth $N$ without requiring hard-coded loops. By maintaining a coordinate vector that "ticks" forward, the system can map every multidimensional index $(i_0, i_1, \ldots, i_n)$ to a linear memory offset using the formula:

$$\text{Offset} = \sum_{d=0}^{n-1} (idx_d \times \text{stride}_d) \tag{1.7}$$

While this method is capable of handling arbitrary rank-N tensors, where the rank is the number of dimensions of the tensor, without pre-compilation, it is limited by the overhead of coordinate-to-offset calculations performed inside the innermost loop. It serves as a study for the genuine implementation of the algorithm that is capable of handling any tensor.

**JIT-Code Generation and Specialization:** Another possible strategy is **Just-In-Time (JIT)** compilation. Instead of calling a library, the engine generates a raw C or C++ source file containing the exact nested loops required for the specific tensor shapes at hand.

By hardcoding the loop bounds and strides, the system allows the compiler to perform aggressive optimizations like **SIMD vectorization** and **Loop**

**Unrolling**. This eliminates the "index lookup" overhead entirely, creating a zero-overhead kernel specialized for a single mathematical task.

**Canonical Permutation and Flattening:** By using a **Permute-Flatten** strategy we can treat the operation as a **matrix multiplication**. This involves reordering the axes of the input tensors so that the summation indices are grouped contiguously.

Once the data is rearranged, the multidimensional tensor is "flattened" into a 2D matrix view. This allows the system to treat a 4D or 6D contraction as a simple 2D `GEMM` operation, drastically reducing the complexity from $O(d^n)$ to $O(M \cdot K \cdot N)$, where $M, K, N$ are the products of the respective dimension groups.

**BLAS Pattern Dispatching:** When treating 1D or 2D patterns (such as Dot, GER, or GEMV), the best solution is to route the data to **Basic Linear Algebra Subprograms (BLAS)**. These are hardware-vendor-tuned kernels (like Intel MKL or Apple Accelerate) that use sophisticated cache-blocking and tiling techniques.

# Chapter 2

# Implementations of the Einstein sum

This chapter presents the different implementations of the Einstein sum.

## 2.1 Generalized Tensor Contraction via Odometer Iteration

This first implementation provides a generalized approach to tensor contraction by treating the dimensions of the tensors as units of measure for an **Odometer**. This object allows the system to systematically iterate through all possible index combinations required by the Einstein summation notation. This method is built upon the principle of explicit summation, where every calculation is performed by manually traversing each coordinate of the tensors involved.

While this approach offers high clarity for educational purposes and flexibility across tensor ranks, its computational complexity grows exponentially. For a tensor of rank $n$ and dimension $d$, the total iterations required is $d^n$, making it a robust "one-size-fits-all" structure primarily suitable for small datasets.

### 2.1.1 The Technical Architecture

The implementation relies on three core components to manage the complexity of multi-dimensional spaces: **Lexicographic Stepping, Bitwise Labeling, and Linear Offsetting**.

**Lexicographic Stepping (next_indices):** The core of the iteration logic is the `next_indices` function. This simulates a multi-dimensional counter that increments in row-major order.

- It identifies the least-significant dimension (the fastest-changing index).
- It increments the index until the maximum bound is reached, at which point it "carries" the increment to the next more-significant dimension.

This logic allows a single flat loop to simulate an arbitrary number of nested loops, enabling the code to handle any tensor rank dynamically.

**Bitwise Labeling (IndexBitmap):** To manage Einstein notation labels (e.g., 'i', 'j', 'k'), the system utilizes an `IndexBitmap`. This converts lowercase letters into a 26-bit integer.

$$\text{Bitmap} = \sum \text{bit}(c - \text{'a'})$$

This allows for nearly instantaneous membership tests. Finding common indices between two tensors is reduced to a simple bitwise `AND` operation, eliminating expensive string comparisons during the inner loops of the summation.

**Linear Offset Calculation:** Data is stored in a contiguous memory buffer following a row-major model. The `compute_offset` function translates a multi-dimensional coordinate into a single memory address:

$$\text{Offset} = \sum_{i=0}^{n-1} \left( \text{index}_i \cdot \prod_{j=i+1}^{n-1} \text{shape}_j \right)$$

While this provides $O(n)$ access time, the dynamic recalculation of these offsets during each iteration step accounts for the primary logic overhead of this approach.

### 2.1.2 Execution and Performance Characteristics

In the final execution stage, the `einsum` function creates a "combined iteration shape" encompassing every unique index in the notation. The odometer sweeps across this high-dimensional space, multiplying values and accumulating them into the result.

| **Feature** | **Odometer Implementation** |
|---|---|
| Computational Efficiency | Moderate (significant logic overhead per iteration) |
| Memory Overhead | Very Low (no temporary tensor copies required) |
| Flexibility | Maximum (handles any rank/notation out-of-the-box) |
| Complexity Scaling | $O(d^n)$ iterations |

**Table 2.1.** Performance characteristics of the Odometer-Based Generalized approach.

### 2.1.3 Summary of the Generalized Engine

The primary strength of this implementation is its **Source Anonymity**. Because it does not rely on hardcoded loops or specific patterns, it acts as an interpreter for Einstein notation.

## 2.2 Just-In-Time (JIT) Code Generation and AOT Specialization

This implementation represents a **Meta-Programming** approach to tensor operations. Instead of relying on a generalized function that interprets notation at runtime, this system acts as a domain-specific compiler. It writes custom-tailored C

source code for specific tensor shapes and notations, which is then compiled into a highly optimized binary kernel.

The primary shift here is the move from runtime interpretation to **Ahead-of-Time (AOT) Specialization**. By baking the geometry of the tensors directly into the instruction stream, we eliminate the CPU cycles typically wasted on managing loop metadata and index validation.

### 2.2.1 Optimization Mechanisms

The generator utilizes three primary optimization strategies: **Literal Stride Injection, Hierarchical Loop Ordering, and Compiler-Assisted Vectorization**.

**Literal Stride Injection:** In generalized functions, strides are looked up in arrays at runtime. This generator calculates these strides once and prints them as literal constants.

For an index "ij" with shape $[7, 3]$, the generator outputs the direct arithmetic `(i * 3) + j`.

**Hierarchical Loop Ordering:** The generator enforces a strict hierarchy to maximize **Data Locality**:

**Outer Shell:** Iterates over the Free Indices (those present in the output), ensuring a contiguous and organized traversal of the destination memory.

**Inner Core:** Iterates over Summation Indices.

This structure maximizes **Temporal Locality** by keeping the accumulation sum for a specific output element within a high-speed CPU register for the entire duration of the summation loop.

**Compiler-Driven Vectorization:** By outputting high-level C code rather than raw assembly, the implementation leverages the full power of modern optimizers.

Because loop bounds (e.g., `i < 7`) are known at compile time, the compiler can perform aggressive **Loop Unrolling** and automatically inject **SIMD (Single Instruction, Multiple Data)** instructions to process multiple floating-point operations in parallel.

### 2.2.2 Performance and Memory Trade-offs

| **Feature** | **JIT Code Generation** |
|---|---|
| Computational Efficiency | Extreme (hardware-level optimization) |
| Memory Overhead | Minimal (no temporary tensors, only source code strings) |
| Flexibility | Low (requires a new function for every shape change) |
| Complexity Handling | Best for static, performance-critical production kernels |

**Table 2.2.** Performance characteristics of the JIT-based specialization approach.

### 2.2.3 Summary of the JIT Engine

This approach treats Einstein notation as a high-level language. By moving the computational burden from the runtime environment to the **Compilation Phase**, it minimizes logic overhead and memory latency, making it adapt for production environments where tensor shapes are known in advance.

## 2.3 Canonical Contraction through Permutation and Flattening

This implementation introduces a transformation designed to bridge the gap between abstract tensor notation and linear algebra. Rather than iterating through a high-dimensional space with a generalized odometer, this approach transforms any arbitrary $N$-dimensional Einstein summation into a standard 2D matrix multiplication ($C = A \times B$).

By reducing the problem to a canonical matrix form, the system can leverage optimized hardware routines while maintaining the flexibility to handle complex tensor geometries.

### 2.3.1 The Transformation Pipeline

The core logic follows a three-stage pipeline: **Permute, Flatten, and Multiply**. This strategy is designed to reduce any arbitrary $N$-dimensional contraction into a single, **GEMM** (General Matrix Multiplication) call.

**Tensor Permutation and Grouping:** To treat a tensor as a matrix, the memory layout must be rearranged so that relevant dimensions are contiguous. The function `matrix_permute` reorders the axes of the source tensors based on their roles in the Einstein notation:

**Tensor A:** Indices are grouped into $[\text{Free Indices}, \text{Summation Indices}]$.

**Tensor B:** Indices are grouped into $[\text{Summation Indices}, \text{Free Indices}]$.

This grouping ensures that the "right side" of Tensor A (the dimensions being summed over) aligns perfectly with the "left side" of Tensor B in linear memory.

**Formal Proof via Matricization:** The validity of this reduction is grounded in the mathematical theory of **unfolding**, or **matricization** [5]. Let $\mathcal{A}$ be a tensor of rank $N$. A matricization is a bijective mapping that reorders the elements of $\mathcal{A}$ into a matrix $A$.

*Proof:* Consider the generalized contraction $\mathcal{C}_{IJ} = \sum_K \mathcal{A}_{IK}\mathcal{B}_{KJ}$. We define a mapping $\phi$ that flattens the multi-index $I$ into a single row index $i$, and $K$ into a column index $k$. If $M = \prod \text{shape}(I)$ and $L = \prod \text{shape}(K)$, then $A \in \mathbb{R}^{M \times L}$ where $A_{\phi(I),\phi(K)} = \mathcal{A}_{IK}$. By performing a similar mapping for $\mathcal{B}$ into $B \in \mathbb{R}^{L \times N}$, the summation becomes:

$$C_{ij} = \sum_{k=1}^{L} A_{ik} B_{kj} \tag{2.1}$$

This expression is identical to the definition of matrix multiplication. Because the mapping $\phi$ is a bijection determined by the tensor's strides, the algebraic integrity of the operation is preserved [6]. □

**Flattening into 2D Matrices:** Once dimensions are grouped, the implementation calculates the product of the shapes in each group to determine the effective number of rows and columns. Let $I$ be the set of free indices in $A$, and $K$ be the set of shared summation indices. Tensor $A$ is treated as a matrix of size $M \times L$, where:

$$M = \prod_{i \in I} \text{shape}(i) \quad \text{and} \quad L = \prod_{k \in K} \text{shape}(k) \tag{2.2}$$

Once the permutation is complete, we pass the resulting memory pointer to the BLAS routine, effectively treating the multidimensional data as a 2D array.

**Multiplication and Reshaping:** With the tensors transformed, the code invokes a BLAS-compliant multiplication:

$$C_{ij} = (A \cdot B)_{ij} \tag{2.3}$$

The result initially exists in a canonical order where all free indices from $A$ appear before those from $B$. A final `matrix_permute` call is performed on the intermediate result to match the specific output notation requested by the user.

### 2.3.2 Performance and Memory Trade-offs

| **Feature** | **Permute-Flatten Implementation** |
|---|---|
| Computational Efficiency | Very High (utilizes linear matrix multiplication) |
| Memory Overhead | Significant (requires temporary permuted copies) |
| Complexity Handling | Excellent for large contractions $O(M \cdot L \cdot N)$ |
| Cache Behavior | Optimized via contiguous linear writes |

**Table 2.3.** Performance characteristics of the Canonical Contraction approach.

### 2.3.3 Summary of the Permute Logic

The helper function `matrix_permute` maps the linear index of the destination tensor back to the coordinates of the source tensor. By iterating through the **destination** linearly, it ensures that memory writes are contiguous.

## 2.4 BLAS-Accelerated Pattern Recognition and Greedy Dispatch

This implementation utilizes a **Greedy Pattern Matching** strategy to map Einstein notation to the highly optimized Basic Linear Algebra Subprograms (**BLAS**) library.

Instead of treating every contraction as a generic tensor operation, this system attempts to identify standard linear algebra archetypes that can be executed using hardware-tuned Level-1, Level-2, or Level-3 BLAS routines.

### 2.4.1 The Pattern Hierarchy

The implementation utilizes a hierarchy of functions, prioritizing specialized kernels over more general ones to minimize computational overhead.

1. **Level-1 Operations (Vector-Vector):**

   **DOT:** Detects patterns like `"i,i->"` to perform a scalar dot product.

   **AXPY:** Recognizes `"i,i->i"` for constant-multiplication and vector addition ($y := \alpha x + y$).

2. **Level-2 Operations (Matrix-Vector):**

   **GER:** Identifies `"i,j->ij"` to perform a rank-1 outer product.

   **GEMV:** Matches `"ij,j->i"` or `"i,ij->j"` to execute matrix-vector multiplication, supporting both standard and transposed variants.

3. **Level-3 Operations (Matrix-Matrix):**

   **GEMM:** The most critical kernel, matching `"ij,jk->ik"`.

### 2.4.2 Intelligent Dispatch Logic

The system analyzes the input notation using bitwise masks and dimension counts to fill a `BLASAnalysis` metadata structure. This structure guides the **Main Dispatch Function**, which routes the matrices to specific case handlers.

| Pattern | BLAS Level | Mathematical Operation |
|---|---|---|
| DOT | Level 1 | Scalar Product: $s = \sum x_i y_i$ |
| AXPY | Level 1 | Vector Accumulation: $y = \alpha x + y$ |
| GER | Level 2 | Outer Product: $A = xy^T$ |
| GEMV | Level 2 | Matrix-Vector: $y = Ax$ |
| GEMM | Level 3 | Matrix-Matrix: $C = AB$ |

**Table 2.4.** The greedy dispatch hierarchy used for BLAS acceleration.

### 2.4.3 Future Work: Stride-Based Optimization

This implementation can be a theoretical groundwork for **Stride-Based Patterns**. While the current version handles unit strides, standard BLAS functions accept an `incx` parameter.

In future iterations, this would allow the system to handle complex patterns like `"ii->"` (Matrix Trace) or `"xii,xii->"` (Diagonal Dot Product) without moving data. By calculating a custom stride based on the tensor shape, the engine could traverse diagonal elements as if they were a contiguous vector, further reducing the need for temporary memory allocations or tensor permutations.

### 2.4.4 Summary of the BLAS Engine

This approach allows us to leverage the BLAS library. By identifying the most specific "building block" for a given operation, we ensure that simple operations remain fast while complex matrix multiplications still take advantage of the optimized BLAS routines.

# Bibliography


[1] George K. Batchelor. *An Introduction to Fluid Dynamics.* Cambridge University Press, 1967.

[2] Morton E. Gurtin. *An Introduction to Continuum Mechanics.* Academic Press, 1981.

[3] Mikhail Itskov. *Tensor Algebra and Tensor Analysis for Engineers.* Springer, 6th edition, 2019.

[4] Julien Klaus, Mark Blacher, and Joachim Giesen. Compiling tensor expressions into einsum. In *International Conference on Conceptual Structures*, 2023.

[5] Tamara G. Kolda and Brett W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3):455–500, 2009. Standard reference for computational tensor contractions and n-mode products.

[6] Stefan Roberts, Spyros Gidaris, Nikos Komodakis, and Patrick Kyriazis. Einsum is all you need. *arXiv preprint arXiv:2010.13454*, 2020. Discusses the equivalence of einsum operations to matricized GEMM operations.

[7] K. Åhlander. Einstein summation for multidimensional arrays. *Computers & Mathematics with Applications*, 44(8):1007–1017, 2002.